



UNIVERSIDAD NACIONAL DE RÍO CUARTO

SISTEMAS OPERATIVOS

(Código 1965)

Taller 3: funciones de utilidad en el kernel

La biblioteca `klib` y la implementación
de `printf` como función variádica

Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Alumno
Tomás Rodeghiero

Fecha de entrega: 17 de mayo de 2026

Índice

Estructura de la entrega	3
La biblioteca klib	3
printf como función variádica	3
Prueba en kernel_main()	5
Compilación y ejecución	5
Salida observada	5
Conclusión	5

Estructura de la entrega

- 03-kernel-printf/: el paso original tal como viene en el .tgz.
- README.md: nota corta con cómo correrlo.
- informe-taller3-kernel-printf.pdf: este informe.

No modifiqué los archivos fuentes; los dejé igual para que se puedan recompilar tal cual.

La biblioteca klib

klib.c junta el conjunto mínimo de funciones que el kernel va a usar:

- `memset`, `memcpy`: manejo básico de memoria.
- `strcpy`, `strcmp`: manejo básico de strings.
- `printf(fmt, ...)`: la pieza central del paso.

La salida ya no se imprime desde `klib.c`: el módulo llama a un `putchar` inline que delega en `console_putc` (`console.c`). Así `klib` queda independiente del dispositivo.

En `klib.h` se definen además dos macros para depurar:

panic y stop en klib.h

```
1  #define panic(fmt, ...) \
2      do { \
3          printf("PANIC: %s:%d: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__); \
4          while (1) {} \
5      } while (0) \
6 \
7 \
8  #define stop() \
9      do { \
10         printf("Kernel stopped at %s:%d\n", __FILE__, __LINE__); \
11         while (1) {} \
12     } while (0) \
```

Las dos imprimen archivo y línea (`__FILE__`, `__LINE__`) y dejan al kernel en un `while(1)`.

printf como función variádica

En C, una *función variádica* es una función con cantidad de argumentos variable. La firma deja al menos un parámetro fijo (acá `fmt`) y el resto se declara con `...`. Cómo se pasan esos argumentos depende de la ABI (registros, pila, o una mezcla), así que para hacerlo portable se usan los macros de `<stdarg.h>`:

- `va_list` `vargs`: representa la lista de argumentos.

- `va_start(vargs, fmt)`: la inicializa apuntando al primer argumento variable.
- `va_arg(vargs, <tipo>)`: devuelve el argumento actual con ese tipo y avanza al siguiente.
- `va_end(vargs)`: limpia la lista al terminar.

`printf` recorre `fmt` y cada vez que ve un `%X` sabe qué tipo pedirle a `va_arg`.

printf en klib.c

```

1 void printf(const char* fmt, ...) {
2     va_list vargs;
3     va_start(vargs, fmt);
4     while (*fmt) {
5         if (*fmt == '%') {
6             fmt++;
7             switch (*fmt) {
8                 case '\0': putchar('%'); goto end;
9                 case '%': putchar('%'); break;
10                case 's': {
11                    const char* s = va_arg(vargs, const char *);
12                    while (*s) putchar(*s++);
13                    break;
14                }
15                case 'd': {
16                    int value = va_arg(vargs, int);
17                    if (value < 0) { putchar('-'); value = -value; }
18                    int divisor = 1;
19                    while (value / divisor > 9) divisor *= 10;
20                    while (divisor > 0) {
21                        putchar('0' + value / divisor);
22                        value %= divisor;
23                        divisor /= 10;
24                    }
25                    break;
26                }
27                case 'x': {
28                    int value = va_arg(vargs, int);
29                    for (int i = 7; i >= 0; i--) {
30                        int nibble = (value >> (i * 4)) & 0xf;
31                        putchar("0123456789abcdef"[nibble]);
32                    }
33                }
34            }
35        } else {
36            putchar(*fmt);
37        }
38        fmt++;
39    }
40    end:
41    va_end(vargs);
42 }

```

Soporta `%s`, `%d` (base 10, con signo), `%x` (8 nibbles, sin 0x) y `%%`. No hay padding, ancho, ni `%u`, `%c` o `%p`, pero para los pasos del EDOS alcanza.

Prueba en `kernel_main()`

El `kernel.c` es muy corto:

`kernel.c`

```
1 #include "klib.h"
2 #include "arch.h"
3
4 void kernel_main(void) {
5     if (cpuid() == 0) {
6         printf("\n\nHello %s\n", "World!");
7         printf("1 + 2 = %d, %x\n", 1 + 2, 0x1234abcd);
8     }
9     stop();
10 }
```

La primera llamada prueba `%s`. La segunda mezcla `%d` y `%x` en el mismo formato, así se ve que `va_arg` avanza bien entre argumentos de distinto tipo. Al final, `stop()` cierra con el mensaje de detención.

Compilación y ejecución

Con el toolchain de Homebrew lo corrí así:

```
1 cd 03-kernel-printf
2 make PREFIX=riscv64-unknown-elf-
3 make qemu PREFIX=riscv64-unknown-elf-
```

Para salir de QEMU: `Ctrl-A` y después `x`.

Salida observada

```
Hello World!
1 + 2 = 3, 1234abcd
Kernel stopped at kernel.c:10
```

Las dos primeras líneas son las dos llamadas a `printf` (`%s`, `%d` y `%x`, este último con los 8 nibbles tal cual). La tercera es la macro `stop()`, que reporta `kernel.c:10`: la línea donde está escrita la llamada en el fuente.

Conclusión

Lo interesante para mí de este taller es ver que, con muy poco código y sin `libc`, alcanza para tener un `printf` útil dentro del kernel. La clave está en los macros de `<stdarg.h>`, que esconden cómo cada ABI pasa los argumentos, más un canal de salida mínimo (la UART, en `console.c`). Las macros `panic` y `stop` completan lo justo para depurar mostrando archivo y línea, que es lo que va a usar el resto de los pasos.